

作業ログ（第7週末）

報告者	櫛濱 和輝 (25G1046)
報告日	2026 年 6 月 9 日
プロジェクト名	赤外線通信で歌うカエル
担当パート	Processing（ピアノ演奏プログラム）／ Arduino（ドラムの通信用関数）
本来の今週の位置づけ	第7週末（子機プログラム実装週）
今週の主担当	210：ピアノのプログラム（Processing 連携）＋ 180：ドラムの通信用関数 <code>hack-drum.ino</code> （ドラム音色 170/171 は久家担当の別タスク）
補足	第6週はログ未提出のため、本ログは第5週末ログ（5/27）からの続きとして記載

1. 進捗サマリー

- 現在地：第7週末（6/9）．実装フェーズの子機プログラム週．第6週はログ未提出だが，第5週末に先行着手していた 110（ピアノ音の作成）はその後完了済み
- 今週は **210：ピアノのプログラム（Processing 連携）**と **180：ドラムの通信用関数（`hack-drum.ino`）**を実装．210 は**実機で動作確認まで完了**したが，180 は**プログラム作成のみで実機テストは未実施**
- 第6週は対面できなかったため検証範囲が限られるが，ピアノ系（親機 → 赤外線 → ピアノ子機 → Processing）では `hack-oya2.ino` と `hack_ko2.pde` を用いて**親機からの譜面送信と発音に成功**
- ドラムは久家と作業を分け，今回は **180：ドラムの通信用関数（`hack-drum.ino`）**を櫛濱が担当（ドラム音色 170/171 の `.pde` 作成は久家担当の別タスク）．本通信用関数は**プログラムを作成したのみで実機テストは未実施**
- 全体進捗率：45%（音色作成フェーズ完了，子機・通信フェーズ進行中）
- 主要成果：
 - － Processing 側（`hack_ko2.pde`）が子機 Arduino からシリアルで届く MIDI 番号を受信し，`PianoInstrument` で発音．MIDI → 周波数変換 $f = 440 \cdot 2^{(m-69)/12}$ を実装
 - － ピアノ系では「親機 → 赤外線（NEC） → ピアノ子機 → USB シリアル → Processing → 発音」の経路が一通貫で動作し，譜面の送信を実機で確認
 - － ドラムの通信用関数（`hack-drum.ino`，課題 180）は NEC フレーム（`address==2`）のみを拾い `command`（MIDI 番号）をシリアルへ中継する設計で実装（自前タイマを持たずテンポは親機任せ）．久家のドラム音色（`.pde`）と組み合わせて鳴らす想定だが，**実機テストは未実施**
- 重大課題（影響度：高）：
 - － ドラムの通信用関数（`hack-drum.ino`，180）が未テスト：第6週に対面できず，久家のドラム音色（`.pde`）と結合した実機での受信・発音確認ができていない

- 音切り替え時のノイズ：現状 `noteOff()` が `unpatch` の即時切断のため、MOP5（フェードアウト）が未対応
- 赤外線通信の成功率（MOP3：95% 以上）が未計測．取りこぼし時の挙動も未検証

2. ガントチャート

計画書 v1.2（2026/5/29 版）でチーム全体のスケジュールが更新されたため、チーム全体・個人の両方について変更前後を併記する．橙色セル（）は前倒し・先行着手した週，水色セル（）は計画通りの実施週を表す．

2.1 チーム全体ガントチャート（変更前：第 5 週末提出版）

管理	第 2 層	第 3 層タスク	担当者	6 週	7 週	8 週	9 週	10 週
110	100 (Processing)	ピアノ音の作成	櫛濱和輝					
120	100 (Processing)	arduino との通信用関数 (ピアノ)	櫛濱和輝					
130	100 (Processing)	フルート音の作成	家田祐希					
140	100 (Processing)	arduino との通信用関数 (フルート)	家田祐希					
150	100 (Processing)	トランペット音の作成	木下遼介					
160	100 (Processing)	arduino との通信用関数 (トランペット)	木下遼介					
170	100 (Processing)	ドラム音の作成 (キック)	久家力孔					
171	100 (Processing)	ドラム音の作成 (シンバル)	久家力孔					
180	100 (Processing)	arduino との通信用関数 (ドラム)	久家力孔					
210	200 (子機)	ピアノのプログラム	櫛濱和輝					
220	200 (子機)	フルートのプログラム	家田祐希					
230	200 (子機)	トランペットのプログラム	木下遼介					

240	200 (子機)	楽譜の設定	渡邊乃斗						
310	300 (親機)	赤外線管理プログラム	渡邊乃斗						
320	300 (親機)	テンポ管理プログラム	渡邊乃斗						
410	400 (その他)	回路作成	手代木巧						
420	400 (その他)	LED マトリクス作成	手代木巧						
510	500 (包括)	結合テスト	全員						

2.2 チーム全体ガントチャート（変更後：計画書 v1.2・第 7 週末時点）

計画書 v1.2 で 5 週列を追加し、先行着手・前倒し分を反映した。171（ドラムシンバル）は新規追加タスク。

管理	第 2 層	第 3 層タスク	担当者	5 週	6 週	7 週	8 週	9 週	10 週
110	100 (Processing)	ピアノ音の作成	櫛濱和輝						
120	100 (Processing)	arduino との通信用関数 (ピアノ)	櫛濱和輝						
130	100 (Processing)	フルート音の作成	家田祐希						
140	100 (Processing)	arduino との通信用関数 (フルート)	家田祐希						
150	100 (Processing)	トランペット音の作成	木下遼介						
160	100 (Processing)	arduino との通信用関数 (トランペット)	木下遼介						
170	100 (Processing)	ドラム音の作成 (キック)	久家力孔						
171	100 (Processing)	ドラム音の作成 (シンバル) 新規	久家力孔						
180	100 (Processing)	arduino との通信用関数 (ドラム)	久家力孔						
210	200 (子機)	ピアノのプログラム	櫛濱和輝						

220	200 (子機)	フルートのプログラム	家田祐希						
230	200 (子機)	トランペットのプログラム	木下遼介						
240	200 (子機)	楽譜の設定	渡邊乃斗						
310	300 (親機)	赤外線管理プログラム	渡邊乃斗						
320	300 (親機)	テンポ管理プログラム	渡邊乃斗						
410	400 (その他)	回路作成	手代木巧						
420	400 (その他)	LED マトリクス作成	手代木巧						
510	500 (包括)	結合テスト	全員						

主な変更点 (計画書 v1.1 → v1.2) :

- 110 (ピアノ音の作成) : 第 6 週予定 → 第 5 週末に先行着手 (その後完了)
- 130 (フルート音) ・ 150 (トランペット音) : 第 6 週予定 → 第 5 週末に前倒し完了
- 160 (トランペット通信用関数) ・ 230 (トランペット子機プログラム) : 第 7 週予定 → 第 5 週末から先行着手
- 171 (ドラム音の作成 ・ シンバル) : 新規追加タスク
- 130 の担当変更 (渡邊 → 家田)

2.3 個人ガントチャート (変更前)

第 5 週末ログ時点の個人計画. 210 (ピアノのプログラム) は第 7 週に割り当てられていた.

管理	第 2 層	第 3 層タスク	5 週	6 週	7 週	8 週	9 週	10 週
110	100 (Processing)	ピアノ音の作成						
210	200 (子機)	ピアノのプログラム						
120	100 (Processing)	arduino との通信用関数 (ピアノ)						
510	500 (包括)	結合テスト (全員)						

2.4 個人ガントチャート (変更後)

第 7 週で 210 (ピアノのプログラム) を完了. 加えて, ドラムは久家と作業を分け, **180 : ドラムの通信用関数 (hack-drum.ino)** を榎濱が担当 (橙色. 音色 170/171 の .pde は久家の別タスク).

Processing 側のシリアル受信処理を実装した関係で、120（通信用関数・ピアノ）の一部も第7週に前倒して着手済み。ただし 180 は第6週に对面できず**実機テストは未実施**（プログラム作成のみ）。

管理	第2層	第3層タスク	5週	6週	7週	8週	9週	10週
110	100 (Processing)	ピアノ音の作成（完了）						
210	200 (子機)	ピアノのプログラム（Processing 連携, 完了）						
180	100 (Processing)	ドラムの通信用関数 <code>hack-drum.ino</code> （音色は久家／未検証）追加						
120	100 (Processing)	arduino との通信用関数（ピアノ）						
510	500 (包括)	結合テスト（全員）						

2.5 今週の作業位置づけ

- 現在地は第7週末（6/9）。本来は子機プログラム（210/220/230）の実装週。
- 予定どおり **210：ピアノのプログラム**（Processing 側でシリアル受信→発音）を実装・動作確認まで完了。
- ピアノは親機 → 赤外線 → ピアノ子機 → Processing の経路で**譜面送信・発音に成功**。
- ドラムは久家と作業を分け、櫛濱が 180：ドラムの通信用関数（`hack-drum.ino`）を実装したが、第6週に对面できず**実機テストは未実施**（音色 170/171 は久家担当）。
- 次週（第8週）はドラム通信用関数の実機テスト、120（通信用関数・ピアノ）の正式化、MOP5（フェードアウト）対応に移る。

3. 作業ログ

作業項目	開始日	予定完了日	状態	進捗率	依存関係・ブロッカー	コメント
ピアノのプログラム (210)	6/3	6/9	完了	100%	子機 Arduino (<code>hack-ko2.ino</code>) / 親機	Processing でシリアル受信した MIDI 番号を周波数へ変換し <code>PianoInstrument</code> で発音。波形描画も実装（付録 9.3.1）

ドラムの通信用関数 (180) hack-drum.ino	6/4	6/9	作成済 (未検証)	実装 100% ／ 検証 0%	親機の NEC 送出／久家のドラム音色 .pde	NEC (address==2) の command をシリアルへ中継する設計で作成. 第 6 週に对面できず実機テスト未実施 (付録 9.3.2)
ピアノ系の統合・動作確認	6/9	6/9	完了	100%	ピアノ子機 hack-ko2.ino / Processing	親機 → IR → ピアノ子機 → Processing の経路で譜面送信・発音を実機確認. ドラムは未テスト (付録 9.3.3)

4. 評価事項

計画書 v1.2 「1.5 テスト項目 (修正版)」のうち、今週の実装 (210 ピアノのプログラム, 180 ドラムの通信用関数, 通信経路) に対応する項目を整理する。

4.1 対応する有効指標 MOE

No.	MOE	評価基準
MOE3	楽器音らしい音色	合成音がピアノとして聴衆に認識される

4.2 対応する性能指標 MOP (計画書 v1.2)

No.	テスト名	内容
MOP1	音色単体テスト (各楽器)	Processing 単体で各楽器音を出力・確認
MOP2	シリアル通信テスト	Arduino-Processing 間通信の正常動作確認
MOP3	赤外線通信テスト	親機→子機の通信成功率 95% 以上
MOP5	フェードアウトテスト 新規	音切り替え時にノイズが生じないこと

MOP6	倍速演奏テスト 新規	スイッチによるテンポ切り替え動作確認
------	------------	--------------------

4.3 今週の自己評価

No.	項目	達成状況	備考
MOE3	楽器音らしい音色	OK	倍音 [1.0, 0.6, 0.35, 0.2, 0.1, 0.05] + ADSR で発音. 聴感でピアノとして認識可
MOP1	音色単体テスト	OK	Processing 単体で C4~A4 の発音を確認
MOP2	シリアル通信テスト	OK	921600bps でピアノ子機 → Processing の MIDI 受信. 取りこぼしなく発音 (ドラム経路は未テスト)
MOP3	赤外線通信テスト	暫定 OK	親機 → ピアノ子機経由で譜面を送信し発音を確認. ドラムの通信用関数は未テスト. 成功率の定量計測も未実施
MOP5	フェードアウト	未対応	<code>noteOff()</code> が即時 <code>unpatch</code> のため切り替えノイズの懸念. 次週対応
MOP6	倍速演奏	未対応	親機側スイッチとの連携が未実装

4.4 今後評価予定の項目 (参考)

- MOP3: 赤外線通信成功率 95% 以上の定量測定 (取りこぼし時の挙動含む)
- MOP4: 音色認識テスト (聴衆過半数が各楽器と識別)
- MOP5: 音切り替え時のフェードアウト実装後に再評価
- MOP6: 親機スイッチ連携後のテンポ切り替え確認

5. 課題管理

課題	発生原因	影響度	優先度	対策	期限	状態
音切り替え時のノイズ	<code>noteOff()</code> が即時 <code>unpatch</code> (MOP5 未対応)	高	高	Line で 0.03 秒フェードアウト後に切断	6/16	未対応

赤外線通信成功率が未計測	MOP3（95% 目標）の定量試験未実施	中	中	一定回数の送受信でログ集計し成功率を算出	6/16	未対応
取りこぼし時の無音・ズレ	ドラム子機は自前タイマを持たず親機任せ	中	中	親機の IR_GAP_MS 調整と再送方針の検討	6/23	未対応

※影響度＝プロジェクトへのインパクト

※優先度＝対応の緊急性

6. AI 利用状況と検証

6.1 利用内容

- 利用目的：Processing 側のシリアル受信→MIDI→周波数変換→発音の実装，およびドラム子機 .ino の NEC 受信処理の実装
- 入力（プロンプト要旨）：
 - 親機が NEC で「address = 子機 ID, command = MIDI 番号」を送る前提を提示し，Processing 側の受信・発音コードを依頼
 - 既存試作（第 5 週の hackathon.pde，音名文字列ベース）を MIDI 番号ベースへ移行する方針を指示
 - IRremote v4.x（NEC）でドラム子機が address==2 のみ拾う最小実装を依頼
- 出力概要：
 - Processing 側 hack_ko2.pde（MIDI 受信→PianoInstrument 発音）
 - ドラム子機 hack-drum.ino（NEC 受信→シリアル中継）

6.2 検証方法

- 検証手法：
 - Processing 単体で MIDI 番号を流し込み，C4～A4 が正しい音高で鳴るか聴感確認
 - ピアノ子機（hack-ko2.ino）と親機（hack-oya2.ino）を接続し，IR 経由で届く譜面で発音されるか実機確認（ドラムの通信用関数は未テスト）
- 参照資料：
 - 計画書・設計書 v1.2（2026/5/29 版）「1.2 PBS」「2.2 詳細設計」
 - Minim ライブラリ公式ドキュメント（ddf.minim.ugens）
 - IRremote ライブラリ（v4.x, NEC プロトコル）
- 検証手順：
 - 親機・各子機・Processing を接続して起動
 - 親機の melody[] と drumPattern[] に沿って IR 送出
 - ピアノ子機が MIDI 番号をシリアルへ中継し，Processing が発音するかを聴感と波形で確

認（ドラムの通信用関数は同手順で次週テスト予定）

6.3 ファクトチェック結果

- 一致点：
 - MIDI → 周波数変換 $f = 440 \cdot 2^{(m-69)/12}$ が標準的な平均律の式と一致
 - シリアル速度 921600bps が設計書「2.1 基本設計の修正」の確定値と一致
 - NEC の address / command の対応（子機 ID / MIDI 番号）が親機実装と整合
- 不一致・疑義：
 - 設計書 2.1 が要求する「noteOff で 0.03 秒フェードアウト後に切断」が未実装（現状は即時 unpatch）. MOP5 の課題として残存
- 最終判断：採用（今週の到達点としては妥当）
- 根拠：
 - 親機 → IR → ピアノ子機 → Processing の経路で譜面送信・発音まで確認でき、ピアノ系の目的を満たしているため（ドラムの通信用関数は次週テスト予定）

7. 次回計画

- 目標（第 8 週）：
 - ドラムの通信用関数（hack-drum.ino, 180）の実機テスト：親機 → ドラム子機 → Processing で、久家のドラム音色（.pde）と結合して受信・発音を確認
 - 120（通信用関数・ピアノ）の正式実装と整理（受信パースの堅牢化）
 - MOP5: noteOff() に Line フェードアウト（0.03 秒）を実装し、切り替えノイズを除去
 - MOP3: 赤外線通信成功率の定量測定（95% 目標に対する実測）
- 達成基準：
 - 音切り替え時にノイズが聞こえないこと
 - 通信成功率を数値で提示できること
 - 他楽器子機と合わせた同期演奏が破綻しないこと
- 期限：
 - 6/16（第 8 週末）

8. その他

- ドラムは通信用関数（櫛濱・180）と音色（久家・170/171）で担当が分かれるため、drumPattern[] の最終仕様と .pde 側の発音をすり合わせ予定
- 親機担当（渡邊）と IR_GAP_MS や倍速演奏（MOP6）スイッチの連携仕様を確認する

9. 付録

9.1 添付資料

- ソースコード：hack_ko2.pde（ピアノ Processing プログラム）, hack-drum.ino（ドラムの通信用関数・180）
- ドラム音色の .pde（170/171）は久家担当のため本ログには含めない
- 参考：hack-oya2.ino（親機・統合用）, hack-ko2.ino（ピアノ子機 Arduino）
- リポジトリ：<https://github.com/k-kushihama/hackathon-code/>

9.2 添付範囲

- 210（ピアノのプログラム）に対応する Processing 側ソースの現時点スナップショット
- 担当したドラムの通信用関数（hack-drum.ino）のソース，および通信経路の構成図・回路図

9.3 ソースコード・図

9.3.1 ピアノ Processing プログラム（hack_ko2.pde）

子機 Arduino からシリアルで届く MIDI 番号を受信し， $f = 440 \cdot 2^{(m-69)/12}$ で周波数へ変換して PianoInstrument で発音する．音色は第 5 週に作成した倍音構造（WavetableGenerator.gen10 の振幅配列 [1.0, 0.6, 0.35, 0.2, 0.1, 0.05]）と Line による ADSR をそのまま用いる．

```
1 import processing.serial.*;
2 Serial port;
3 import ddf.minim.*;
4 import ddf.minim.ugens.*;
5 Waveform currentWaveform;
6 Minim minim;
7 AudioOutput out;
8 int currentNote = 0;
9
10 void setup() {
11     size(512, 200);
12     minim = new Minim(this);
13     out = minim.getLineOut();
14     out.setTempo(120);
15     currentWaveform = WavetableGenerator.gen10(4096, new float[] { 1.0f, 0.6f, 0.35f, 0.2
16         f, 0.1f, 0.05f });
17     port = new Serial(this, "/dev/cu.usbmodem34B7DA64C6002", 921600);
18     port.clear();
19     port.bufferUntil('\n');
20 }
21
22 void draw() {
23     background(0);
24     stroke(255);
25     for (int i = 0; i < out.bufferSize() - 1; i++) {
```

```

25     line(i, 50 + out.left.get(i)*50, i+1, 50 + out.left.get(i+1)*50);
26 }
27 fill(255);
28 text("note: " + currentNote, 10, 180);
29 }
30
31 void serialEvent(Serial p) {
32     String inString = p.readStringUntil('\n');
33     if (inString != null) {
34         inString = trim(inString);
35         if (inString.length() == 0) return;
36         int midi = int(inString);
37         currentNote = midi;
38         if (midi > 0) {
39             playNote(midi);
40         }
41     }
42 }
43
44 void playNote(int midi) {
45     float freq = (float)(440.0 * Math.pow(2.0, (midi - 69) / 12.0));
46     out.playNote(0, 0.4f, new PianoInstrument(freq, 0.6f, currentWaveform));
47 }
48
49 class PianoInstrument implements Instrument {
50     Oscil wave;
51     Line ampEnv;
52     Oscil vib;
53     Summer freqSum;
54     Constant baseFreq;
55     float maxAmp;
56
57     PianoInstrument(float frequency, float maxAmp, Waveform wf) {
58         this.maxAmp = maxAmp;
59         freqSum = new Summer();
60         baseFreq = new Constant(frequency);
61         vib = new Oscil(5, 0, Waves.SINE);
62         baseFreq.patch(freqSum);
63         vib.patch(freqSum);
64         wave = new Oscil(frequency, 0, wf);
65         freqSum.patch(wave.frequency);
66         ampEnv = new Line();
67         ampEnv.patch(wave.amplitude);
68     }
69
70     void noteOn(float duration) {
71         ampEnv.activate(duration, this.maxAmp, 0);
72         wave.patch(out);
73     }

```

```

74
75 void noteOff() {
76     wave.unpatch(out);
77 }
78 }

```

9.3.2 ドラムの通信用関数 (hack-drum.ino・課題 180)

ドラムは作業を分担し、本プログラム（通信用関数）を櫛濱が、ドラム音色の .pde (170/171) を久家が担当する。赤外線受光モジュールで NEC フレームを受信し、自分宛 (CHILD_ID = 2) のものだけを拾って command (MIDI 番号) をシリアルへ中継する。楽譜を持たずステートレスで、テンポはすべて親機任せのため可変 BPM に自動追従する設計だが、第 6 週に対面できず**実機テストは未実施**。

```

1  #define DECODE_NEC
2  #include <IRremote.hpp>
3
4  #define CHILD_ID 2
5
6  const int PIN_IR_RECV = 2;
7  const int LED_INDICATOR = 13;
8
9  void setup() {
10     Serial.begin(921600);
11     IrReceiver.begin(PIN_IR_RECV);
12     pinMode(LED_INDICATOR, OUTPUT);
13 }
14
15 void loop() {
16     if (IrReceiver.decode()) {
17         if (IrReceiver.decodedIRData.protocol == NEC) {
18             uint16_t address = IrReceiver.decodedIRData.address;
19             if (address == CHILD_ID) {
20                 uint8_t note = (uint8_t)IrReceiver.decodedIRData.command;
21                 Serial.println(note);
22                 digitalWrite(LED_INDICATOR, HIGH);
23                 delay(15);
24                 digitalWrite(LED_INDICATOR, LOW);
25             }
26         }
27         IrReceiver.resume();
28     }
29 }

```

9.3.3 システム統合：データフロー

図 1 に、親機から発音までのデータフローを示す。親機 (hack-oya2.ino) が melody[] / drumPattern[] に沿って NEC フレーム (address = 子機 ID, command = MIDI 番号) を送

出し、各子機は自分宛のフレームだけを拾って MIDI 番号をシリアルへ中継、Processing が受信して発音する。ピアノ子機は `CANON_OFFSET_TICKS` 分だけ遅れて開始することで輪唱（カノン）を実現する。本ログ時点では、このうちピアノ系（親機 → ピアノ子機 → Processing）の譜面送信・発音までを実機で確認しており、ドラム経路（`hack-drum.ino` + 久家のドラム音色 `.pde`）は未テストである。



図1 システム統合のデータフロー（親機 → IR/NEC → 子機 → USB シリアル → Processing → 発音）

9.3.4 ドラム子機 Arduino の回路図

図2にドラム子機 Arduino の接続を示す。赤外線受光モジュール CHQ1838D を1つ用い、親機からの NEC フレームを受信する。D2 に受光信号、D13 に動作確認用 LED を接続し、USB を Processing 実行用 PC へつなぐ。

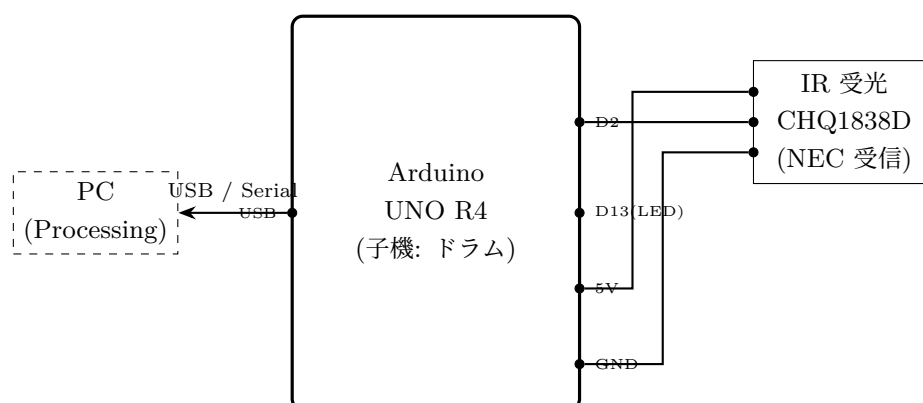


図2 ドラム子機 Arduino の回路図。D2 に IR 受光モジュール、D13 に動作確認 LED を接続し、USB 経由で Processing 実行用 PC と通信する。

なお、赤外線受光モジュールは内部にプルアップを持つため、Arduino 側は `INPUT_PULLUP` 相当で十分。電源は Arduino の 5V / GND を共用する。